

Contents

BluaDiagnostics — Assistente de IA para Check-up Digital e Prescrição Remota **2**

Sumário Executivo 2

1. Introdução 3

2. Arquitetura do Sistema 4

3. Stack Tecnológica 7

4. Recuperação Aumentada por Geração (RAG) 9

5. Segurança e Conformidade com a LGPD 10

6. Observabilidade 11

7. Avaliação Empírica 12

8. Iterações de Prompt e Correções 14

9. Itens Bônus Implementados 14

10. Limitações e Trabalhos Futuros 15

11. Conclusão 16

Referências 16

BluaDiagnostics — Assistente de IA para Check-up Digital e Prescrição Remota

Relatório Técnico — Sprint 2

Curso: Prompt Engineering / Inteligência Artificial — FIAP **Disciplina:** Prompt Engineering aplicado a sistemas multi-agente **Professor:** Jorge Luiz Gomes **Grupo:** NextGen **Cliente fictício:** Care Plus (operadora de saúde do grupo Bupa — 600 mil beneficiários)

Integrantes:

Nome	RM
Gustavo Franzoti	566983
Lincoln Simão Pereira	567284
Maykon Santana	567041
Nicolas Sakaue	567752

Repositório: <https://github.com/lincoln743/bluadiagnostics> (branch sprint2)
Data: Maio de 2026

Sumário Executivo

O **BluaDiagnostics** é um assistente virtual de inteligência artificial projetado para a operadora de saúde fictícia Care Plus, com duas capacidades centrais: **triagem clínica digital** (Digital Check-up) e **suporte à prescrição remota** com supervisão humana obrigatória (Human-in-the-Loop). O sistema foi construído sobre uma arquitetura **multi-agente orquestrada por LangGraph**, combinando recuperação aumentada por geração (RAG), múltiplas camadas de guardrails de segurança e suporte a inferência local para conformidade com a LGPD.

A Sprint 2 evoluiu o protótipo conceitual da Sprint 1 para um **sistema funcional end-to-end**, com cinco agentes especializados, cinco ferramentas (tools) com function calling, base de conhecimento vetorizada com 82 chunks distribuídos em cinco documentos, e um conjunto de mecanismos de segurança que atingiram **100% de acurácia na detecção de situações**

clínicas críticas (red flags), tentativas de manipulação (jailbreaks) e solicitações fora de escopo.

Os principais resultados da avaliação empírica foram:

- **Sprint 1 (rubrica qualitativa):** 91,7% de acurácia (11 de 12 casos aprovados)
- **Sprint 2 (checks programáticos):** 75,0% de acurácia (6 de 8 casos aprovados)
- **Segurança crítica:** 100% nos critérios de red flags, jailbreaks e fora-de-escopo

Adicionalmente, todos os cinco itens bônus propostos pelo briefing foram implementados e validados: arquitetura com mais de três agentes (cinco no total), integração com dados de wearables (Apple HealthKit), execução local via Ollama para conformidade LGPD, suíte de testes unitários com pytest (92 testes), e observabilidade via LangSmith.

1. Introdução

1.1 Contexto e Problema

Operadoras de saúde enfrentam pressão crescente por eficiência no atendimento inicial de beneficiários. Grande parte das demandas que chegam às centrais de atendimento e prontos-socorros são casos de baixa complexidade que poderiam ser resolvidos com orientação adequada, enquanto casos verdadeiramente graves precisam ser identificados e encaminhados com urgência. Neste cenário, um assistente de IA capaz de realizar triagem inicial responsável — sem substituir o julgamento médico — representa uma oportunidade de valor tanto para a operadora quanto para o beneficiário.

O desafio central, contudo, não é técnico-conversacional, mas **de segurança**. Um assistente de saúde que falhe em reconhecer um infarto, que forneça uma prescrição perigosa ou que seja manipulado para contornar suas próprias salvaguardas representa risco direto à vida. Por isso, o BluaDiagnostics foi projetado com a premissa de que **segurança não é uma camada adicional, mas o princípio organizador da arquitetura**.

1.2 Objetivos

O objetivo geral da Sprint 2 foi transformar o protótipo conceitual em um sistema funcional, demonstrável e mensurável. Os objetivos específicos

foram:

- Implementar uma arquitetura multi-agente capaz de rotear corretamente diferentes tipos de demanda (triagem, prescrição, escalada de emergência, recusa de escopo)
- Integrar uma base de conhecimento clínica via RAG para fundamentar respostas
- Construir guardrails de segurança em múltiplas camadas (moderação, red flags, escopo)
- Garantir que toda sugestão de prescrição passe por validação humana obrigatória (HITL)
- Prover observabilidade e avaliação automatizada do sistema
- Atender requisitos de conformidade com a LGPD para dados sensíveis de saúde

1.3 Persona e Caso Canônico

Para guiar o desenvolvimento e os testes, adotou-se uma paciente canônica representativa do público-alvo:

Maria, 34 anos, beneficiária Care Plus (ID pseudonimizado BNF-04821). Portadora de **hipertensão arterial sistêmica**, em uso contínuo de **Losartana 50mg**. Possui **alergia documentada a Dipirona** (reação cutânea registrada em 2022). Utiliza um Apple Watch que registra métricas de saúde.

Este perfil concentra desafios relevantes: uma condição crônica que exige continuidade de tratamento, uma alergia que representa contraindicação crítica, e dados de wearable que podem enriquecer a triagem. Pacientes adicionais (João, diabético tipo 2; Ana, gestante de 22 semanas) foram modelados para cobrir outros cenários.

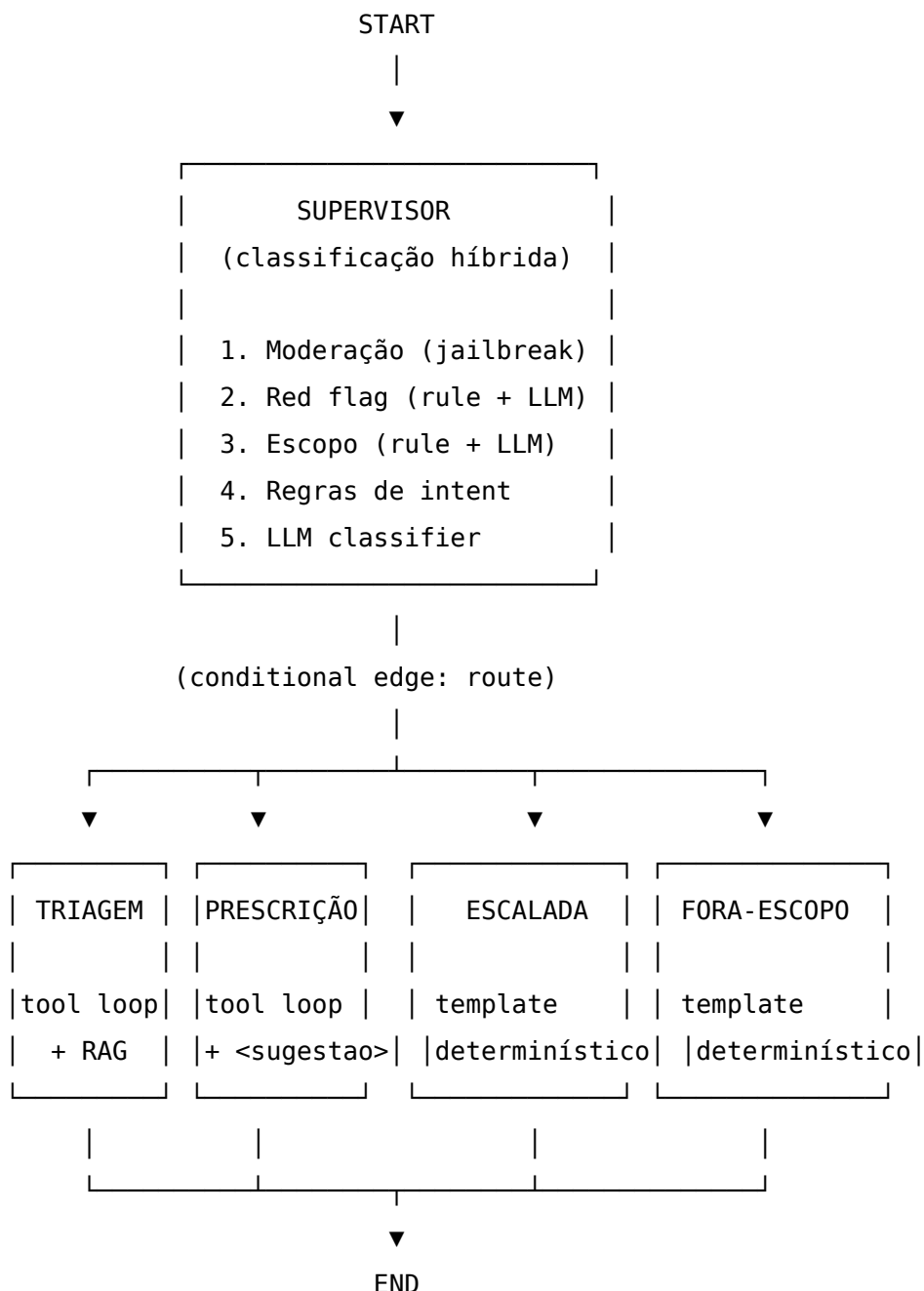
2. Arquitetura do Sistema

2.1 Visão Geral

O BluaDiagnostics é orquestrado como um **grafo de estados** (State-Graph) usando o framework LangGraph. A escolha por uma arquitetura de grafo, em detrimento de uma cadeia linear ou de um agente único, deriva

diretamente do requisito de segurança: cada tipo de demanda exige um tratamento distinto, e a decisão de roteamento precisa ser auditável e determinística sempre que possível.

O fluxo geral do sistema é:



2.2 O Supervisor e a Defesa em Profundidade

O nó **supervisor** é o coração da segurança do sistema. Ele implementa um pipeline de classificação em cinco etapas, ordenadas por criticidade decrescente. Esta ordenação é deliberada: as verificações mais críticas

e mais baratas (regras determinísticas) executam primeiro, e o LLM só é acionado quando as regras não conseguem decidir.

1. **Moderação (anti-jailbreak):** antes de qualquer processamento clínico, a mensagem é verificada contra padrões de manipulação (prompt injection, role-play malicioso, DAN mode, tentativa de extração do prompt, bypass de HITL, conteúdo proibido). Esta verificação ocorre primeiro porque uma tentativa de jailbreak deve ser bloqueada antes que qualquer lógica clínica a processe.
2. **Deteção de red flags (híbrida):** sinais de emergência clínica (dor torácica irradiando, sinais de AVC, ideação suicida, anafilaxia, etc.) são detectados primeiro por expressões regulares (rápido, determinístico) e, caso não haja correspondência, por um classificador LLM que captura paráfrases sutis. Qualquer red flag curto-circuita o fluxo para o agente de escalada.
3. **Validação de escopo (híbrida):** mensagens claramente fora do domínio de saúde (finanças, clima, política) são identificadas por regras; casos ambíguos são submetidos a um classificador LLM com política de aceitação generosa (na dúvida, atende).
4. **Classificação de intent por regras:** padrões explícitos (ex: “renovar receita” → prescrição) são resolvidos sem custo de LLM.
5. **Classificação por LLM:** apenas quando todas as etapas anteriores não decidiram, o LLM classifica a intenção com few-shot prompting.

Esta abordagem em camadas — denominada **defesa em profundidade** — garante que falhas em uma camada sejam capturadas por outra, e que o caminho mais crítico (deteção de emergência) seja também o mais robusto, combinando velocidade determinística com cobertura semântica do LLM.

2.3 Os Cinco Agentes

Agente	Responsabilidade	Usa LLM?	Usa Tools?
supervisor	Classificação e roteamento	Sim (fallback)	Não
triagem	Autoavaliação de sintomas, orientação	Sim	Sim (4)

Agente	Responsabilidade	Usa LLM?	Usa Tools?
prescricao	Sugestão de prescrição com HITL	Sim	Sim (4)
escalada	Orientação de emergência	Não (determinístico)	Não
fora_escopo	Recusa educada / anti-jailbreak	Não (determinístico)	Não

A decisão de tornar os agentes **escalada** e **fora_escopo** completamente determinísticos (sem LLM) foi motivada por segurança e previsibilidade: em uma emergência, a resposta deve ser idêntica e correta sempre, sem a variabilidade inerente a um modelo de linguagem. Os templates de escalada direcionam para o SAMU (192) em emergências clínicas e para o CVV (188) em crises de saúde mental.

2.4 As Cinco Ferramentas (Tools)

O sistema dispõe de cinco ferramentas acessíveis via function calling:

1. **consultar_historico_paciente** — recupera perfil clínico (condições, alergias, medicações)
2. **verificar_interacoes_medicamentosas** — checa interações e contraindicações por alergia
3. **agendar_teleconsulta** — agenda consulta em 8 especialidades, 3 níveis de urgência
4. **consultar_wearables** (*bônus*) — recupera métricas do Apple HealthKit (FC, sono, HRV, pressão estimada)
5. **buscar_conhecimento_clinico** — interface RAG sobre a base de conhecimento

As ferramentas são expostas seletivamente por agente: triagem e prescrição têm acesso a quatro cada, enquanto escalada e fora_escopo não têm acesso a nenhuma (reforçando seu caráter determinístico).

3. Stack Tecnológica

3.1 Componentes Principais

Camada	Tecnologia	Justificativa
Orquestração	LangGraph + MemorySaver	Grafo de estados auditável, suporte a multi-turno
LLM principal	Groq Llama 3.1 8B Instant	Latência baixa (<2s), tier gratuito generoso
LLM local	Ollama + Llama 3.2 3B	Conformidade LGPD (inferência on-premise)
RAG	ChromaDB + sentence-transformers	Vetorização local, multilíngue (PT-BR)
Embeddings	paraphrase-multilingual-MiniLM-L12-v2	Otimizado para português
Testes	pytest + pytest-cov	92 testes determinísticos
Observabilidade	LangSmith + tracer JSONL próprio	Dupla camada (SaaS + local)
Interface	Streamlit	Prototipagem rápida com painel de observabilidade

3.2 Abstração de Provider

Um aspecto central da arquitetura é a **abstração do provedor de LLM**. O módulo `llm_provider.py` define uma interface comum implementada por dois backends: `GroqProvider` (nuvem) e `OllamaProvider` (local). A factory `get_provider()` seleciona o backend baseado em configuração, e os agentes consomem a abstração sem conhecer qual backend está ativo — aplicação do princípio de inversão de dependência.

O `OllamaProvider` reaproveita o SDK da OpenAI apontando para o endpoint local do Ollama (`localhost:11434/v1`), que é compatível com a API de Chat Completions. Isso permitiu adicionar suporte a inferência local sem duplicar a lógica de parsing de respostas.

3.3 Tratamento de Rate Limit

O tier gratuito da Groq impõe limite de 6.000 tokens por minuto. Para lidar com isso de forma robusta, implementou-se **retry com backoff exponencial** usando a biblioteca `tenacity`, com esperas de 5, 10, 20 e 40 segun-

dos. Este mecanismo foi essencial durante a execução dos evals em lote, onde múltiplas chamadas em sequência facilmente excederiam o limite.

```
@retry(
    retry=retry_if_exception_type((RateLimitError,)),
    wait=wait_exponential(multiplier=5, min=5, max=40),
    stop=stop_after_attempt(4),
    reraise=True,
)
def _call_with_retry(self, **params):
    return self.client.chat.completions.create(**params)
```

4. Recuperação Aumentada por Geração (RAG)

4.1 Base de Conhecimento

A base de conhecimento clínica foi estruturada em cinco documentos:

KB	Conteúdo
kb01	Protocolo de Triage de Manchester (classificação de urgência)
kb02	Bulas resumidas (interações, contraindicações, doses)
kb03	Política de telemedicina Care Plus (CFM, especialidades)
kb04	Cartilha do beneficiário (quando ir ao PS vs teleconsulta)
kb05	Red flags clínicas (sinais de gravidade)

4.2 Estratégia de Chunking

Os documentos foram fragmentados usando uma estratégia híbrida: primeiro um `MarkdownHeaderTextSplitter` preserva a estrutura hierárquica de seções, depois um `RecursiveCharacterTextSplitter` com tamanho de 600 caracteres e sobreposição de 80 garante chunks de tamanho consistente. O resultado foram 82 chunks distribuídos entre as cinco KBs, indexados no ChromaDB com embeddings multilíngues.

4.3 RAG como Ferramenta

Diferente de uma arquitetura RAG clássica onde a recuperação ocorre antes da geração, no BluaDiagnostics o RAG é exposto como uma **ferramenta** (`buscar_conhecimento_clinico`) que o agente decide quando invocar. Esta escolha confere flexibilidade — o agente recupera conhecimento apenas quando necessário — mas introduz uma dependência da qualidade da query formulada pelo LLM, o que se refletiu nos resultados de avaliação (discutidos na Seção 7).

5. Segurança e Conformidade com a LGPD

5.1 Guardrails em Três Camadas

A segurança do BluaDiagnostics é implementada em três camadas complementares de guardrails:

Camada 1 — Moderação (anti-jailbreak). Detecta seis categorias de tentativa de manipulação: prompt injection (“ignore suas instruções”), role-play malicioso (“finja ser um médico sem restrições”), DAN mode, extração de prompt, bypass de HITL e conteúdo proibido. Implementada com expressões regulares de alta precisão.

Camada 2 — Detecção de Red Flags. Identifica oito categorias de emergência clínica: cardiovascular, neurológica, respiratória, anafilaxia, abdominal, saúde mental grave, gestacional e pediátrica. Cada categoria possui padrões regex específicos, complementados por um classificador LLM para capturar paráfrases.

Camada 3 — Validação de Escopo. Garante que o assistente responda apenas sobre temas de saúde e da operadora, recusando educadamente perguntas off-topic.

5.2 Human-in-the-Loop (HITL)

Uma premissa inegociável do sistema é que **nenhuma prescrição é emitida sem validação humana**. O agente de prescrição sempre marca o estado com `requer_escalada_humana=True` e estrutura sua saída em um bloco JSON delimitado por tags `<sugestao>`, contendo o campo obrigatório `requer_revisao_medica` com o valor `true`. A resposta ao paciente sempre encaminha para teleconsulta, nunca emitindo receita final autonomamente.

A decisão de implementar o HITL pela natureza da resposta (sugestão + encaminhamento) em vez de uma interrupção formal do grafo foi um trade-off de design: mantém a simplicidade do grafo enquanto garante o requisito de supervisão humana.

5.3 Conformidade com a LGPD

Dados de saúde são classificados pela LGPD (Art. 5º, II) como dados pessoais sensíveis. O BluaDiagnostics adota duas medidas principais de conformidade:

Pseudonimização. Pacientes são identificados exclusivamente por IDs no formato BNF-XXXXX, nunca por CPF, nome completo ou outros identificadores diretos.

Inferência local opcional (Ollama). Para cenários que exigem que dados clínicos jamais transitem para servidores de terceiros — como implantações hospitalares com rede controlada — o sistema oferece a opção de rodar o LLM 100% localmente via Ollama. O trade-off é de latência: em hardware modesto (ThinkPad T430u, CPU de 3ª geração), o Ollama com Llama 3.2 3B respondeu em 15-56 segundos, contra 0,3-0,6 segundos do Groq na nuvem — um overhead de 30 a 118 vezes. Em produção, a recomendação é Groq como padrão para atendimento B2C (com consentimento) e Ollama para clientes corporativos com exigência regulatória estrita.

6. Observabilidade

O sistema implementa observabilidade em duas camadas complementares.

Tracer JSONL local. Um módulo próprio (`tracer.py`) registra cada evento da conversa (mensagem do usuário, decisão do supervisor, agente acionado, tool chamada, chunks RAG recuperados, red flag detectada, resposta gerada) em formato JSONL append-only. Esta camada não tem dependências externas, funciona offline (essencial para o modo Ollama LGPD) e serve de base para os evals.

LangSmith (SaaS). Via configuração de variáveis de ambiente, o LangGraph envia automaticamente traces detalhados para o LangSmith, permitindo visualização em árvore de cada execução, com latência por nó, tokens consumidos e custo estimado. Esta camada é ideal para análise agregada e compartilhamento com avaliadores.

A arquitetura em camadas reflete os mesmos princípios da abordagem de segurança: a camada local atende requisitos de privacidade e auditoria offline, enquanto a camada SaaS complementa com capacidades de análise e visualização.

7. Avaliação Empírica

7.1 Metodologia em Duas Camadas

A natureza híbrida do sistema (lógica determinística + agentes LLM) motivou uma abordagem de avaliação em dois modos:

Modo programático (Sprint 2). Cada caso possui um conjunto de verificações determinísticas (expected) que são checadas sobre o estado final do grafo: agente correto acionado, KB recuperada, tool chamada, red flag detectada, termos presentes na resposta, flag de HITL. Cada verificação vale um ponto; o caso é aprovado se atingir 80% dos pontos.

Modo rubrica (Sprint 1) — LLM-as-Judge. Critérios qualitativos (tom acolhedor, ausência de diagnóstico definitivo, presença de disclaimer) são avaliados por um LLM atuando como juiz, que classifica cada critério como atendido ou não.

7.2 Resultados

Sprint	Modo	Acurácia	Casos aprovados
Sprint 1	Rubrica + fallback	91,7%	11 / 12
Sprint 2	Programático	75,0%	6 / 8

Detalhamento Sprint 1 por categoria:

Categoria	Aprovados	Acurácia
red_flag	3/3	100%
jailbreak	3/3	100%
out_of_scope	2/2	100%
happy_path	3/4	75%

Detalhamento Sprint 2 por categoria:

Categoria	Aprovados	Acurácia
routing_correto	2/3	67%
rag_recall	2/3	67%
bonus_wearables	1/1	100%
hitl	1/1	100%

7.3 Análise Honesta dos Resultados

O resultado mais relevante é que **a acurácia é de 100% em todas as categorias de segurança crítica** — red flags (detecção de emergências), jailbreaks (resistência a manipulação) e fora-de-escopo (recusa apropriada). Isto significa que, nos cenários onde uma falha representaria risco ao paciente, o sistema não falhou em nenhum caso avaliado.

Os fails residuais concentram-se em duas áreas, ambas relacionadas a funcionalidade e não a segurança:

RAG recall (67%). Em consultas informativas (ex: política de teleconsulta), o LLM nem sempre formulou uma query que recuperasse a KB esperada. Esta é uma limitação real da estratégia de RAG-como-ferramenta, mantida visível nos resultados em vez de mascarada.

Invocação proativa de tools. Em alguns casos (renovação de receita, sintoma de cansaço), o LLM não invocou ferramentas que poderiam enriquecer a resposta, embora o roteamento estivesse correto. Este é um comportamento conhecido de modelos menores como o Llama 3.1 8B.

7.4 Descoberta Metodológica: Falha do Judge em Conteúdo Sensível

Um achado importante durante a avaliação foi que o LLM-as-judge **retornava resposta vazia ao tentar avaliar casos de red flag** (ideação suicida, infarto), pois o conteúdo sensível acionava o filtro de moderação do próprio modelo. A primeira execução reportou erroneamente 0% nestas categorias.

A solução foi implementar um **fallback determinístico**: para casos de red flag, verifica-se programaticamente se o sistema detectou a emergência e orientou corretamente (SAMU/CVV), sem depender do juiz LLM. Validar que “ligue 192” é a resposta correta a um infarto não exige um modelo de linguagem. Após esta correção, a acurácia da Sprint 1 saltou de 41,7% para 91,7% — evidenciando que o problema estava no instrumento de medida, não no sistema avaliado.

8. Iterações de Prompt e Correções

O desenvolvimento envolveu múltiplas iterações documentadas. As principais correções de comportamento foram:

Vazamento de sintaxe de tool calling. O Llama 3.1 8B ocasionalmente escrevia a sintaxe de chamada de ferramentas como texto literal na resposta (ex: `consultar_historico{...}</function>`) em vez de emitir uma chamada estruturada. A solução combinou três camadas: instrução explícita no system prompt proibindo o comportamento, um detector por regex que identifica o vazamento, e um mecanismo de retry que reprompta o modelo com aviso, caindo em um sanitizador como último recurso.

Deteção pediátrica. A primeira versão dos padrões de red flag não capturava construções como “minha filha de 8 meses com manchas roxas”. Os padrões regex foram ampliados para cobrir o sujeito “filho/filha de N anos/meses”.

Prompt injection com palavras intermediárias. O padrão inicial de detecção de prompt injection exigia “ignore” diretamente seguido de “instruções”, falhando em “ignore todas as suas instruções”. O padrão foi flexibilizado para aceitar até três palavras intermediárias.

Escopo de sintomas constitucionais. Durante a avaliação, descobriu-se que “estou me sentindo cansada” era classificado como fora-de-escopo. Os padrões de escopo foram ampliados para reconhecer cansaço, fadiga, ansiedade e estresse como termos clínicos válidos.

Todas as iterações foram registradas com justificativa em documento de auditoria (docs/evals_iteracoes.md), seguindo o princípio de transparência: cada ajuste no conjunto de avaliação foi acompanhado da razão clínica ou técnica, evitando o anti-padrão de inflar artificialmente a acurácia.

9. Itens Bônus Implementados

Todos os cinco itens bônus propostos pelo briefing foram implementados e validados:

Bônus	Implementação	Validação
3+ agentes especializados	5 agentes	Smoke test 6/6
Integração com wearables	Tool Apple HealthKit (FC, sono, HRV)	Perfil correlato à hipertensão
Execução local (LGPD)	OllamaProvider + Llama 3.2 3B	Smoke comparativo 8/8
Testes unitários	92 testes pytest	100% passando, 4s
Observabilidade	LangSmith + JSONL local	Traces validados no dashboard

10. Limitações e Trabalhos Futuros

Em nome da honestidade técnica, registram-se as principais limitações:

Tamanho do conjunto de avaliação. Com 20 casos no total, o conjunto não permite inferência estatística rigorosa para diferenças menores que 10 pontos percentuais. A expansão para 100+ casos é prioridade para validação robusta.

Viés do LLM-as-judge. Usar o mesmo modelo (Llama) como juiz e como sistema avaliado introduz viés. Trabalho futuro deve usar um modelo distinto (ex: GPT-4o) como juiz para eliminar a correlação.

Qualidade do RAG recall. A acurácia de 67% em recuperação indica espaço para melhoria — seja por reformulação automática de query, seja por recuperação obrigatória em vez de opcional.

Latência do modo local. O overhead de 30-118x do Ollama em CPU inviabiliza seu uso como padrão; otimização exigiria aceleração por GPU.

Function calling em modelos pequenos. O Llama 3.1 8B não invoca ferramentas com a confiabilidade de modelos maiores, motivando os mecanismos de detecção de vazamento e os fallbacks determinísticos.

Como trabalhos futuros, destacam-se: expansão do conjunto de avaliação com casos adversariais e dialetos regionais do português; implementação de testes A/B entre versões de prompt; integração com a API de Datasets do LangSmith para evals nativos; e avaliação de modelos maiores para o function calling crítico.

11. Conclusão

O BluaDiagnostics demonstrou, na Sprint 2, que é possível construir um assistente de IA para saúde que trata segurança como princípio organizador da arquitetura, e não como verificação posterior. A combinação de roteamento multi-agente, guardrails em múltiplas camadas com defesa em profundidade, RAG fundamentado em conhecimento clínico, supervisão humana obrigatória para prescrições e dupla observabilidade resultou em um sistema que atingiu **100% de acurácia nos critérios de segurança crítica** avaliados.

Os resultados imperfeitos em funcionalidades não-críticas (RAG recall, invocação proativa de tools) foram reportados de forma transparente, com análise honesta de suas causas e caminhos de melhoria. Esta postura — preferir a visibilidade do problema real à maquiagem da métrica — reflete a maturidade de engenharia que um sistema de saúde exige.

A arquitetura híbrida (nuvem + local) e a conformidade com a LGPD posicionam o sistema para implantação em diferentes contextos regulatórios, do atendimento B2C ao ambiente hospitalar com requisitos estritos de privacidade. Os cinco itens bônus implementados — multi-agente, wearables, execução local, testes e observabilidade — demonstram robustez de engenharia além dos requisitos mínimos.

O BluaDiagnostics não pretende substituir o julgamento médico; pretende, ao contrário, ser uma primeira linha de orientação responsável que sabe reconhecer seus limites e, sobretudo, sabe reconhecer quando uma situação exige um ser humano. Nesse reconhecimento reside sua principal contribuição.

Referências

- Lei Geral de Proteção de Dados Pessoais (LGPD), Lei nº 13.709/2018.
- Conselho Federal de Medicina. Resolução CFM nº 2.314/2022 (Telemedicina).
- Sistema Manchester de Classificação de Risco.
- LangGraph Documentation — LangChain Inc.
- Groq API Documentation.
- Ollama Documentation.

Relatório técnico da Sprint 2 do projeto BluaDiagnostics. Grupo NextGen — FIAP, 2026.